

Databases

Backend Development

Relational Schema Design, Applied to Real Systems

Designing a schema for a real backend means translating the actual business rules into tables and constraints — not just entities, but the rules that govern them. A student belongs to exactly one college (a foreign key, not nullable if the rule is strict); a course can have many enrollments, but an enrollment always belongs to exactly one student and one course (a many-to-one relationship from each side). Getting these constraints right at the database level — not just checking them in application code — means the database itself refuses to store impossible data, which is a far stronger guarantee than a check that could be accidentally skipped somewhere in the code.

Query Optimization Basics

Slow queries usually trace back to one of a few common causes: missing indexes on columns used in WHERE/JOIN/ORDER BY, fetching more columns or rows than actually needed (SELECT * instead of the specific fields used), or the classic N+1 query problem — fetching a list of records, then running a separate query per record to get related data, instead of one query that joins or batches the related data upfront. Most ORMs have a way to eagerly load related data in a single query specifically to avoid this N+1 pattern, and recognizing it is one of the highest-value debugging skills for backend performance.

ORMs & Migrations

Beyond writing queries, ORMs manage the evolution of a database schema over time through migrations — versioned, ordered files that each describe one change (add a column, create a table, add an index). Migrations are checked into version control alongside application code, so the database schema's history is tracked the same way code changes are, and any environment (a new developer's machine, staging, production) can be brought to the exact same schema state by running the same ordered set of migrations. Never modifying a production schema by hand, outside of the migration system, is one of the most important habits for keeping environments consistent.

Learn More

MySQL Official Documentation

<https://dev.mysql.com/doc/>

Prisma Official Documentation

<https://www.prisma.io/docs>

PostgreSQL Official Documentation

<https://www.postgresql.org/docs/>